

# MZ2POL-04: Policies and Boundaries

---

**Series:** MZ2POL | **Notebook:** 5 of 8 | **Created:** December 2025 | **Last Updated:** 01/28/2026

## Overview

---

This notebook provides a comprehensive guide to **IAM Policies** and **Policy Boundaries** - the two components that together replace Management Zone access control. Policies define **WHAT** users can do, while boundaries define **WHERE** those permissions apply.

## Prerequisites

---

- Completed MZ2POL-01 through MZ2POL-03
- Access to Dynatrace Account Management
- Policy Management permissions

## Learning Objectives

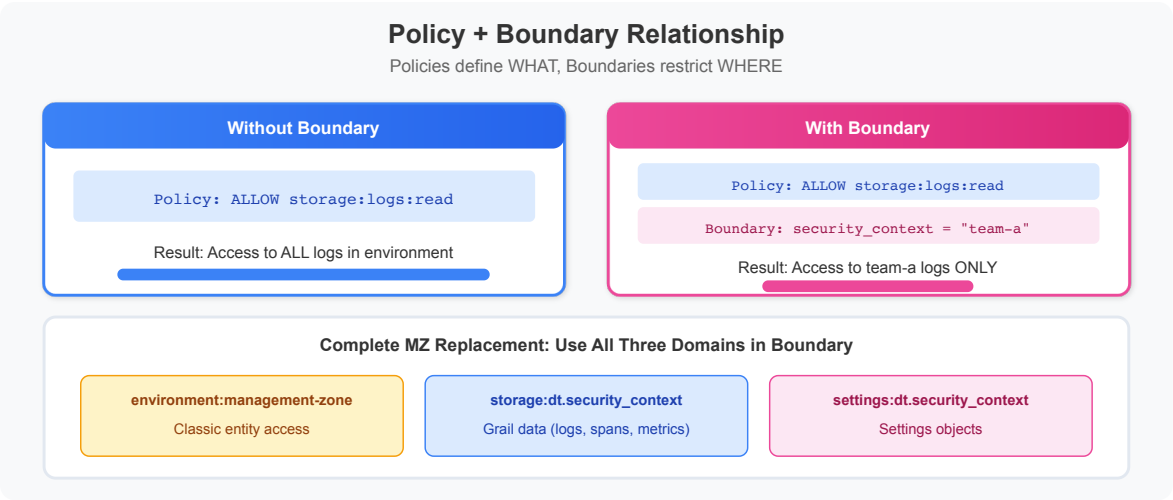
---

By the end of this notebook, you will:

1. Understand policy statement syntax and structure
  2. Know the default policies and when to use them
  3. Understand boundary syntax and how boundaries restrict policies
  4. Be able to map MZ access patterns to policy + boundary combinations
  5. Know the recommended group-policy-boundary structure for SAML/AD integration
-

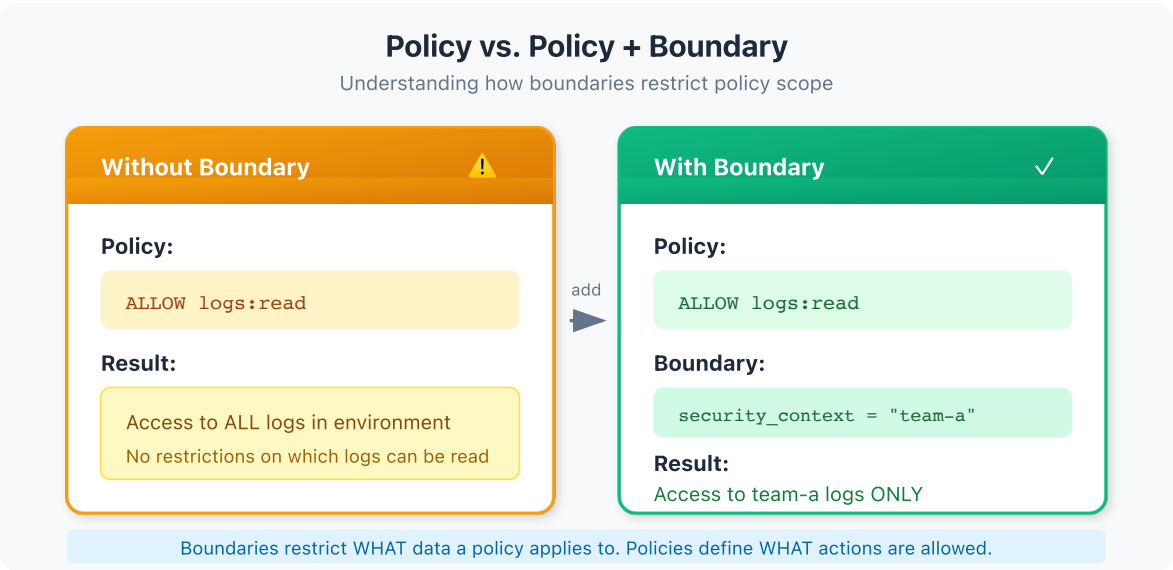
# 1. Understanding the Relationship

## Policies vs Boundaries



| Component | Purpose                          | Analogy                         |
|-----------|----------------------------------|---------------------------------|
| Policy    | Defines WHAT actions are allowed | "You can read logs"             |
| Boundary  | Defines WHERE policies apply     | "...but only for team-frontend" |

## How They Work Together



## Key Characteristics

### Policies:

- Define permissions (ALLOW statements)
- Can include conditions (WHERE clauses)
- Default policies cover most use cases

### Boundaries:

- Separate from policies (decoupled)
- Reusable across multiple policies
- Can only narrow permissions, never expand
- Optional - policies work without them (full scope)

---

## 2. Policy Fundamentals

### Policy Statement Structure

```
<effect> <service>:<resource>:<action> [WHERE  
<condition>]
```

| Component        | Description       | Examples                      |
|------------------|-------------------|-------------------------------|
| <b>Effect</b>    | Permission type   | ALLOW (only option currently) |
| <b>Service</b>   | Dynatrace service | storage , settings , app      |
| <b>Resource</b>  | Resource type     | logs , buckets , objects      |
| <b>Action</b>    | Operation         | read , write , delete         |
| <b>Condition</b> | Optional filter   | WHERE field = "value"         |

## Basic Policy Statements

```
// Grant read access to all logs
ALLOW storage:logs:read

// Grant read/write to buckets
ALLOW storage:buckets:read
ALLOW storage:buckets:write

// Grant all actions on a resource
ALLOW storage:logs:*
```

## Common Services and Resources

| Service    | Resources                                 | Common Actions                |
|------------|-------------------------------------------|-------------------------------|
| storage    | logs , spans , events , metrics , buckets | read , write , delete         |
| settings   | objects , schemas                         | read , write , delete         |
| app        | apps , functions                          | run , install                 |
| automation | workflows                                 | read , write , run            |
| document   | documents                                 | read , write , delete , share |

## 3. Default Policies

Dynatrace provides **default policies** that cover common access patterns. These are **read-only** but cover most use cases.

## Dynatrace Access Policies

| Policy                      | Use Case         | Key Permissions                       |
|-----------------------------|------------------|---------------------------------------|
| Dynatrace Viewer            | Read-only access | View dashboards, data, settings       |
| Dynatrace Operator          | Basic operations | Viewer + create dashboards, notebooks |
| Dynatrace Standard User     | Regular users    | Operator + modify some settings       |
| Dynatrace Professional User | Power users      | Standard + advanced features          |
| Dynatrace Admin User        | Administrators   | Full platform access                  |

## Data Access Policies

| Policy      | Use Case            | Key Permissions             |
|-------------|---------------------|-----------------------------|
| Data Viewer | Read monitored data | Query logs, metrics, traces |
| Data Editor | Modify data configs | Viewer + data configuration |

## Choosing the Right Default Policy

| User Type          | → Dynatrace Policy       | + Data Policy |
|--------------------|--------------------------|---------------|
| Executives/Viewers | → Dynatrace Viewer       | + Data Viewer |
| Developers         | → Dynatrace Standard     | + Data Viewer |
| SRE/Operations     | → Dynatrace Professional | + Data Editor |
| Platform Admins    | → Dynatrace Admin        | + Data Editor |

# 4. Condition and Boundary Syntax

Policies and boundaries share the same condition syntax.

## Supported Operators

| Operator                | Description                 | Example                                |
|-------------------------|-----------------------------|----------------------------------------|
| <code>=</code>          | Exact match                 | <code>field = "value"</code>           |
| <code>!=</code>         | Not equal                   | <code>field != "value"</code>          |
| <code>startsWith</code> | Prefix match                | <code>field startsWith "prefix"</code> |
| <code>in</code>         | Value in list               | <code>field in ("a", "b", "c")</code>  |
| <code>IN</code>         | Value in list (alternative) | <code>field IN ("a", "b")</code>       |

## Supported Fields

| Field                                     | Description               | Used In    |
|-------------------------------------------|---------------------------|------------|
| <code>environment</code>                  | Environment restrictions  | Boundaries |
| <code>environment:name</code>             | Environment by name       | Boundaries |
| <code>environment:management-zone</code>  | MZ-based (transitional)   | Boundaries |
| <code>storage:dt.security_context</code>  | Security context          | Both       |
| <code>storage:bucket</code>               | Grail bucket              | Both       |
| <code>settings:schemaId</code>            | Settings schema ID        | Policies   |
| <code>settings:dt.security_context</code> | Settings security context | Boundaries |

## Combining Conditions

**AND logic** (all conditions must match):

```
ALLOW storage:logs:read
WHERE storage:dt.security_context = "team-a"
AND storage:bucket = "production_logs"
```

**OR logic** (multiple statements or lines):

```
// In policies: Multiple statements
ALLOW storage:logs:read WHERE storage:dt.security_context = "team-a"
ALLOW storage:logs:read WHERE storage:dt.security_context = "team-b"

// In boundaries: Each line is OR-combined
storage:dt.security_context = "team-a"
storage:dt.security_context = "team-b"
```

---

## 5. Creating Boundaries

---

### Via UI

1. Navigate to **Account Management** → **Identity & Access Management**
2. Select **Policy Boundaries** → **Boundaries** tab
3. Click **Create boundary**
4. Enter:
  - **Boundary name:** Descriptive name
  - **Boundary query:** Restriction conditions
5. Click **Save**

### Boundary Examples

#### Team-Based Boundary:

```
Name: Frontend Team Scope
Query: storage:dt.security_context = "team-frontend"
```

#### Environment Boundary:

```
Name: Production Only
Query: storage:dt.security_context startsWith "prod-"
```

#### Multi-Team Boundary (OR logic):

```
Name: Platform Teams
Query:
storage:dt.security_context = "team-infra"
storage:dt.security_context = "team-sre"
storage:dt.security_context = "team-platform"
```

## Boundary Limitations

| Limitation             | Description                     | Workaround                      |
|------------------------|---------------------------------|---------------------------------|
| Max 10 lines           | Only 10 conditions per boundary | Create multiple boundaries      |
| No AND in lines        | Each line is one condition      | Use multiple boundaries for AND |
| No complex expressions | Limited to basic operators      | Simplify conditions             |

## 6. Best Practice: Complete Boundary for MZ Restriction

When restricting access to replicate a Management Zone, use **all three domains** in your boundary query:

```
environment:management-zone IN ("LOB5");
storage:dt.security_context IN ("LOB5");
settings:dt.security_context IN ("LOB5");
```

### Why All Three Domains?

| Domain                                    | What It Restricts                                  |
|-------------------------------------------|----------------------------------------------------|
| <code>environment:management-zone</code>  | Classic entity access (hosts, services, processes) |
| <code>storage:dt.security_context</code>  | Grail data (logs, spans, metrics, events)          |
| <code>settings:dt.security_context</code> | Settings objects                                   |



Using only one domain leaves gaps in access control. For complete MZ replacement, include all three.

---

## 7. Recommended Group-Policy-Boundary Structure

---

For organizations using SAML/SSO with Active Directory, this structure provides clear separation of duties:

```
Group: "LOB5-Team" (SAML from Active Directory)
├─ Policy: Dynatrace Viewer (standard read access)
├─ Policy: Dynatrace Professional
│   └─ Boundary:
│       environment:management-zone IN ("LOB5");
│       storage:dt.security_context IN ("LOB5");
│       settings:dt.security_context IN ("LOB5");
```

### Key Benefits

- **SAML Group:** Ties to existing AD group membership - no separate user management
- **Standard User Policy:** Provides baseline access without boundary (if needed)
- **Pro User + Boundary:** Advanced permissions restricted to team's scope

### Alternative: All Policies with Boundary

For stricter isolation, apply the boundary to all policies:

```
Group: "LOB5-Team" (SAML from AD)
├─ Policy: Dynatrace Viewer
│   └─ Boundary: LOB5-Scope
├─ Policy: Dynatrace Professional
│   └─ Boundary: LOB5-Scope
```

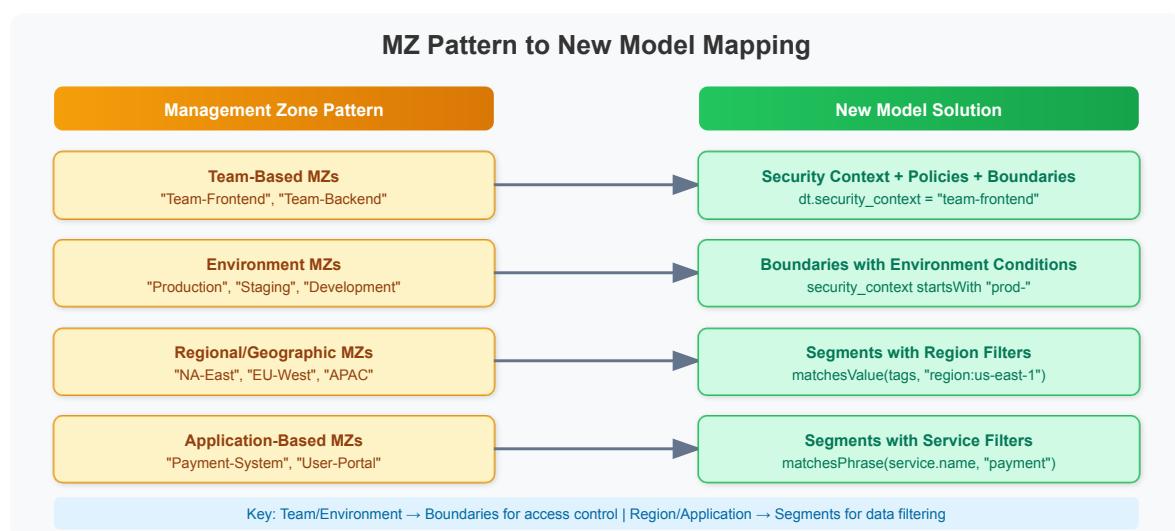
### Binding via UI

1. Navigate to **Group Management**
2. Select or create a group

3. Go to **Permissions** tab
4. Click **Add permission**
5. Select:
  - **Policy**: Choose policy to bind
  - **Boundary** (optional): Select restriction
  - **Environment**: Select target environment
6. Click **Save**

## 8. Mapping MZ Access to Policies + Boundaries

### Common MZ Permission Patterns



| MZ Permission     | Equivalent Policy + Boundary           |
|-------------------|----------------------------------------|
| View only         | Dynatrace Viewer + Boundary            |
| View + Edit       | Dynatrace Standard User + Boundary     |
| Full access to MZ | Dynatrace Professional User + Boundary |
| Admin in MZ       | Custom policy + Boundary               |

## Migration Example: Team-Based MZ

### Before (MZ-based):

- Management Zone: "Frontend-Team"
- Rules: Services with tag `team:frontend`
- Users assigned via RBAC

### After (Policy + Boundary):

1. Set security context on entities:
  - Services get `dt.security_context = "team-frontend"`
2. Create boundary:
  - Name: Frontend Team Scope
  - Query:  
`environment:management-zone IN ("Frontend-Team");`  
`storage:dt.security_context IN ("team-frontend");`  
`settings:dt.security_context IN ("team-frontend");`
3. Bind to group:
  - Group: Frontend Developers (SAML)
  - Policy: Dynatrace Standard User
  - Boundary: Frontend Team Scope

## Migration Example: Environment MZ

### Before (MZ-based):

- Management Zone: "Production"
- Rules: Hosts with tag `env:production`
- Users get MZ-filtered view

### After (Policy + Boundary):

```
1. Set security context on entities:
  - Hosts get dt.security_context = "prod-{region}"

2. Create boundary:
  Name: Production Environment
  Query:
  environment:management-zone IN ("Production");
  storage:dt.security_context startsWith "prod-";
  settings:dt.security_context startsWith "prod-";

3. Bind to group:
  Group: Production Operators (SAML)
  Policy: Dynatrace Professional User
  Boundary: Production Environment
```

---

## 9. Security Context and Bucket Strategy

---

### What Is Security Context?

**Security Context** ( `dt.security_context` ) is an entity attribute used for access control. It's the recommended way to scope boundaries for Grail data.

### Primary Grail Fields

When migrating from Management Zones, leverage **Primary Grail Fields** for consistent data partitioning:

| Field                           | Source          | MZ Migration Use             |
|---------------------------------|-----------------|------------------------------|
| <code>k8s.cluster.name</code>   | Kubernetes      | Replace cluster-based MZs    |
| <code>k8s.namespace.name</code> | Kubernetes      | Replace namespace-based MZs  |
| <code>aws.account.id</code>     | AWS metadata    | Replace AWS account MZs      |
| <code>dt.host_group.id</code>   | OneAgent config | Replace host group-based MZs |

# Bucket Strategy for Migration

**Buckets** partition Grail data and integrate with policies and boundaries for team-level access control.

## Bucket Limitations

 **Critical:** Plan buckets carefully during migration - these constraints cannot be changed later.

| Limitation               | Details                                                        |
|--------------------------|----------------------------------------------------------------|
| One data type per bucket | Logs, metrics, events, OR spans - not mixed                    |
| Names are immutable      | Bucket names CANNOT be changed after creation                  |
| No data migration        | Data CANNOT be moved between buckets                           |
| Naming rules             | 3-100 chars, lowercase alphanumeric, underscores, hyphens only |
| Maximum buckets          | 80 per environment (default limit)                             |
| Optimal ingest           | ~1 TB/day per bucket for best query performance                |
| Acceptable ingest        | 1-3 TB/day per bucket (limited query window)                   |
| Maximum ingest           | 3 TB/day per bucket hard limit                                 |

## Query Constraints

| Limit                    | Value  | Impact                               |
|--------------------------|--------|--------------------------------------|
| Maximum data scanned     | 500 GB | Limits queryable time window         |
| Maximum records returned | 1,000  | Use aggregations for larger datasets |
| Maximum response payload | 1 MB   | Large result sets may be truncated   |

## Default Bucket Retentions

| Bucket          | Retention |
|-----------------|-----------|
| default_logs    | 35 days   |
| default_metrics | 15 months |
| default_spans   | 10 days   |
| default_events  | 35 days   |

## Using Buckets for Team Isolation (MZ Replacement)

When replacing team-based MZs, use buckets in **both policies AND boundaries**:

```
// Policy: Grant team access to their specific buckets
ALLOW storage:logs:read WHERE storage:bucket.name = "frontend_logs"
ALLOW storage:spans:read WHERE storage:bucket.name = "frontend_spans"
ALLOW storage:metrics:read WHERE storage:bucket.name = "frontend_metrics"

// Boundary: Restrict to team's buckets and security context
storage:bucket.name IN ("frontend_logs", "frontend_spans",
"frontend_metrics");
storage:dt.security_context IN ("team-frontend");
environment:management-zone IN ("Frontend-Team");
```

## Migration Mapping: MZ to Buckets

| MZ Pattern                                 | Bucket Strategy                                       |
|--------------------------------------------|-------------------------------------------------------|
| <b>Team MZ</b> (e.g., "Frontend-Team")     | Team-specific buckets: frontend_logs , frontend_spans |
| <b>Environment MZ</b> (e.g., "Production") | Environment buckets: prod_logs , prod_metrics         |
| <b>Regional MZ</b> (e.g., "US-East")       | Region buckets: useast_logs , useast_spans            |
| <b>Compliance MZ</b> (e.g., "PCI")         | Compliance buckets: pci_logs , pci_events             |

## Complete Team Migration Example

### Before (MZ-based):

- Management Zone: "Checkout-Team"
- Users see all data within the MZ

### After (Policy + Bucket + Boundary):

```
Group: Checkout-Team (SAML from AD)
├─ Policy: Checkout Data Access
│   ├── ALLOW storage:logs:read WHERE storage:bucket.name = "checkout_logs"
│   ├── ALLOW storage:spans:read WHERE storage:bucket.name = "checkout_spans"
│   └─ ALLOW storage:metrics:read WHERE storage:bucket.name =
"checkout_metrics"
└─ Boundary:
    storage:bucket.name IN ("checkout_logs", "checkout_spans",
"checkout_metrics");
    storage:dt.security_context IN ("checkout");
    environment:management-zone IN ("Checkout-Team");
```

For detailed bucket design guidance, see **ORGNZ-03: Bucket Strategy and Design** which covers naming conventions, retention planning, and cost attribution patterns in depth.

## Setting Security Context

Security context can be set via:

- **Entity Enrichment rules** (recommended)
- **Host properties** via OneAgent
- **OpenPipeline processing**
- **Auto-tagging rules** (legacy)

## Security Context Naming Strategy

| Pattern       | Example        | Use Case               |
|---------------|----------------|------------------------|
| team-{name}   | team-frontend  | Team ownership         |
| env-{name}    | env-production | Environment separation |
| region-{code} | region-us-east | Geographic isolation   |
| app-{name}    | app-checkout   | Application scoping    |
| {env}-{team}  | prod-frontend  | Combined scoping       |

## Query Entities by Security Context

```
// List services with their security context
// Identify services that need security context assignment
fetch dt.entity.service
| fields entity.name,
        dt.security_context,
        tags,
        managementZones
| sort dt.security_context asc
| limit 50
```

```
// Count entities by security context
// Helps verify security context distribution
fetch dt.entity.service
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

---

## 10. Custom Policies

### When to Create Custom Policies

- Default policies are too broad or narrow
- Need specific permission combinations
- Require conditional access



- Compliance requirements

## Custom Policy Examples

### Log Viewer for Specific Team:

```
// Policy Name: Frontend Team Log Access
// Description: Read-only access to frontend service logs

ALLOW storage:logs:read
  WHERE storage:dt.security_context = "team-frontend"

ALLOW storage:spans:read
  WHERE storage:dt.security_context = "team-frontend"
```

### Environment-Restricted Admin:

```
// Policy Name: Development Admin
// Description: Full access to development environment only

ALLOW storage:*:*
  WHERE storage:dt.security_context startsWith "dev-"

ALLOW settings:objects:*
  WHERE settings:scope startsWith "environment:dev"
```

### Dashboard and Notebook Creator:

```
// Policy Name: Dashboard Creator
// Description: Create and manage dashboards and notebooks

ALLOW document:documents:read
ALLOW document:documents:write
ALLOW document:documents:delete
ALLOW document:documents:share

// Read data for dashboards
ALLOW storage:logs:read
ALLOW storage:metrics:read
ALLOW storage:spans:read
```

---

# 11. Best Practices

---

## Do's

-  **Start with default policies** - customize only when needed
-  **Use least privilege** - grant minimum required permissions
-  **Document policies and boundaries** - clear names and descriptions
-  **Test before production** - verify with test users
-  **Use boundaries for scope** - keep policies reusable
-  **Include all three domains** in boundaries for complete MZ replacement
-  **Use SAML groups** tied to AD for team management

## Don'ts

-  **Don't create policy per user** - use groups
-  **Don't duplicate default policies** - extend instead
-  **Don't use overly broad wildcards** - be specific
-  **Don't embed conditions in policies** when boundaries are better
-  **Don't forget the settings domain** in boundaries

## Naming Conventions

```
// Good names
"Frontend Team - Standard Access"    (policy)
"Production Environment"             (boundary)
"L0B5-Scope"                        (boundary)

// Bad names
"Policy 1"
"Test"
"John's boundary"
```

## Audit Checklist

- ☐ Each boundary has a clear purpose
- ☐ Boundary names follow naming convention
- ☐ All three domains included (environment, storage, settings)

- [ ] SAML groups aligned with AD structure
- [ ] Dependencies tracked (which policies use which boundaries)
- [ ] Regular review scheduled

---

## 12. Troubleshooting

---

### Common Issues

| Issue                        | Likely Cause                  | Solution                         |
|------------------------------|-------------------------------|----------------------------------|
| User can't access anything   | No policy bound               | Bind policy to user's group      |
| User sees too much           | Boundary too broad or missing | Tighten boundary conditions      |
| User can't see expected data | Missing data policy           | Add Data Viewer/Editor           |
| Permissions inconsistent     | Multiple conflicting policies | Review all bound policies        |
| Works in classic, not Grail  | Only environment domain used  | Add storage domain to boundary   |
| Settings not restricted      | Missing settings domain       | Add settings:dt.security_context |

### Debugging Steps

1. **Verify group membership:** Is user in correct group?
2. **Check policy bindings:** What policies are bound to group?
3. **Review boundaries:** Are all three domains included?
4. **Test with admin:** Does admin see the data?
5. **Check security context:** Is it set on the entities?

## Testing Boundaries

1. **Create test user** in target group
  2. **Log in as test user**
  3. **Verify access:**
    - Can access expected data?
    - Cannot access restricted data?
  4. **Test edge cases:**
    - Entities at boundary edges
    - New entities without security context
- 

## Summary

---

In this notebook, you learned:

1. **Policies define WHAT**, boundaries define **WHERE**
2. **Default policies** cover most use cases - customize only when needed
3. **Shared syntax** for conditions in both policies and boundaries
4. **Three domains** needed for complete MZ replacement (environment, storage, settings)
5. **SAML + Policy + Boundary** structure for AD integration
6. **Migration patterns** for team-based and environment-based MZs

## Next Steps

---

Continue to **MZ2POL-05: Segments Implementation** to:

- Create DQL-based data filters
- Replace MZ data filtering with Segments
- Configure cross-app filtering

## Additional Resources

---

- [Working with Policies](#)

- [IAM Policy Reference](#)
  - [Policy Boundaries Documentation](#)
  - [Grant Access with Security Context](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*